

HaluGuard: Execution-Grounded Contrastive Context Selection for Hallucination-Free Repository-Level Code Generation

Sparsh Gupta Alan Liang Katherine Hancock Daniel Ho Freddie Liang

University of Southern California

{sparshg, aqliang, kxhancoc, hodk, fhliang}@usc.edu

Abstract. We report progress on **HaluGuard**, a framework that reframes repository-level context selection as a hallucination prevention problem. Our core contribution is the **Hallucination-Contrastive Context Scorer (HCCS)**, a lightweight scorer trained with a listwise cross-entropy ranking objective on gold-snippet supervision from RepoBench v1.1 [1]. We pre-compute frozen CodeBERT and UniXcoder embeddings and compare four scorer architectures across both encoders (8 experiments), finding that a simple dual-encoder with learned temperature outperforms deeper alternatives. On a held-out test set (70/15/15 split), HCCS achieves 23.4% R@1 and 0.436 MRR, outperforming cosine similarity (+4.6% R@1) and BM25 (+5.8% R@1). On hard examples (≥ 10 candidates), HCCS achieves 18.1% Acc@1 vs. 10.7% for cosine—a +69% relative improvement. Ablation studies on regularization (5 configs) and ranking-focused losses (4 configs) confirm that embedding quality, not model capacity, is the performance bottleneck. Next steps include end-to-end generation evaluation and stronger code encoders.

1. Changes from Original Proposal

Our original proposal planned to use **CodeHaluEval** [2] as the primary evaluation benchmark. Upon closer analysis, we found that CodeHaluEval consists of *standalone function-level coding problems* (699 tasks, 8,883 samples): each task is entirely self-contained with no multi-file repository context and therefore *no cross-file retrieval candidates to rank*. This makes CodeHaluEval fundamentally unsuitable for evaluating a context *selection* system, where the core task is to identify the most relevant snippet from a set of candidate repository files.

We pivoted to **RepoBench v1.1** [1], which provides repository-level cross-file next-line prediction with per-example candidate context snippets and `gold_snippet_index` oracle labels. This benchmark directly supports retrieval evaluation and provides a clean training signal. Table 1 summarizes the key changes.

2. Tasks Performed

2.1 Dataset and Embeddings

We use RepoBench v1.1 Python `cross_file_first` (8,033 examples, ~ 11 can-

Aspect	Proposal	Current
Dataset	CodeHaluEval	RepoBench v1.1
Train obj.	Pairwise contrast	Listwise cross-entropy
Architectures	Single MLP	4 architectures compared
Eval. split	Val = test (leak)	70/15/15 train/val/test
Eval. style	Internal val.	RepoBench baselines
Backbone	CodeBERT only	CodeBERT + UniXcoder
Query repr.	Full context	Last-3-lines + full

Table 1: Key changes from the original proposal.

didate chunks per query). We fixed a data leakage issue in the original pipeline by implementing a proper 70/15/15 train/val/test split, with validation used exclusively for early stopping. For each example we pre-compute frozen 768-d CLS-token embeddings using CodeBERT [4] and UniXcoder [5].

2.2 HCCS Model Training

From pairwise to listwise training. The original HCCS scorer was trained with pairwise InfoNCE loss, comparing one positive chunk against one sampled negative per query. This peaked at 59.7% pairwise accuracy with UniXcoder and $\sim 50\%$ with CodeBERT (random chance). Pairwise training

solves an artificial task—comparing two chunks in isolation—rather than ranking all ~ 11 candidates simultaneously. We switched to **listwise cross-entropy**: the model sees all candidate chunks for a query and the loss is standard cross-entropy with the gold index as the target:

$$\mathcal{L} = -\log \frac{\exp(s(q, c_{\text{gold}}))}{\sum_{j=1}^n \exp(s(q, c_j))} \quad (1)$$

where $s(q, c)$ is the raw scorer logit for the (q, c) pair.

Model architectures. We implemented and compared four scorer architectures, each taking frozen 768-d embeddings as input:

- **DualEncoder** (394K params): Separate query/chunk linear projections ($768 \rightarrow 256$) with LayerNorm and GELU, scored by cosine similarity with a learned log-temperature.
- **DualEncoderDeep** (1M params): Two-layer projections ($768 \rightarrow 512 \rightarrow 256$) with the same cosine scoring.
- **ListwiseMLP** (394K params): Concatenation of $[q; c; |q - c|; q \odot c]$ through a $3072 \rightarrow 512 \rightarrow 1$ feedforward network with raw logits.
- **PairwiseMLP** (394K params): Same concatenation architecture as ListwiseMLP but trained with InfoNCE loss (baseline from prior work).

All four architectures were trained with both CodeBERT and UniXcoder embeddings (8 experiments total). Training used AdamW ($\text{lr} = 5 \times 10^{-4}$, weight decay 10^{-4}), cosine annealing schedule, gradient clipping, and early stopping (patience 20) on validation MRR.

Architecture comparison. Table 2 reports validation results. UniXcoder embeddings dominate CodeBERT across all architectures by ~ 5 MRR points. The simplest DualEncoder wins despite having the fewest parameters, suggesting embedding quality matters more than scorer complexity.

2.3 Baseline Comparison (Test Set)

Table 3 shows evaluation on the held-out test set (never seen during training or early stopping). The best HCCS model (UniXcoder + DualEncoder) improves over cosine similarity by +4.6% R@1 and +3.4 MRR points.

Encoder	Model	R@1	R@5	MRR
UniXcoder	DualEncoder	26.2%	72.2%	0.463
UniXcoder	DualEncoderDeep	25.9%	72.4%	0.459
UniXcoder	ListwiseMLP	25.9%	70.7%	0.457
UniXcoder	PairwiseMLP	25.2%	71.1%	0.450
CodeBERT	DualEncoderDeep	21.0%	66.5%	0.412
CodeBERT	DualEncoder	20.9%	67.2%	0.410

Table 2: Architecture comparison on validation set (top 6 of 8 shown). Best per encoder in bold.

Method	R@1	R@5	MRR
HCCS (DualEncoder)	23.4%	70.7%	0.436
Cosine similarity	18.8%	68.3%	0.402
BM25	17.6%	63.8%	0.378
Random	15.8%	61.0%	0.358

Table 3: Test set evaluation: HCCS vs. baselines (UniXcoder embeddings).

2.4 Easy vs. Hard Evaluation

We split test examples by difficulty following Re-poBench’s protocol: *easy* (5–9 candidates, $n=343$) and *hard* (≥ 10 candidates, $n=562$). Table 4 compares five methods.

The most significant result is on **hard examples**: HCCS achieves 18.1% Acc@1 versus 10.7% for cosine—a **+69% relative improvement**. Notably, the prior pairwise HCCS actually *underperformed* cosine on hard@1 (17.4% vs. 18.3%); the switch to listwise training with the DualEncoder architecture fixes this.

Method	Easy ($n=343$)			Hard ($n=562$)		
	@1	@3	@5	@1	@3	@5
Random	12.8	42.3	75.2	8.2	19.0	31.7
Jaccard	15.5	46.9	74.6	6.8	21.7	35.1
BM25	17.5	49.3	77.3	5.5	19.9	36.5
Cosine	17.2	51.6	79.6	10.7	31.7	44.7
HCCS	23.3	51.9	80.8	18.1	35.2	49.3

Table 4: Easy/hard bucket evaluation (UniXcoder DualEncoder). HCCS wins all columns, especially hard@1 (+69% over cosine).

2.5 Ablation Studies

The model overfits severely (train R@1 = 97%, test R@1 = 23%). We ran 5 regularization configs (higher dropout, weight decay, label smoothing, smaller projections, embedding dropout, and a combined setting). Regularization reduces the train–test

gap from 71.6% to 36.7%, but **no config improved test metrics**. We also tested 4 R@1-focused loss variants (focal loss, hard example mining, top- k contrastive). Focal loss achieved the best validation R@1 (26.8%) but baseline cross-entropy still won on test (MRR 0.444 vs. 0.437). Both ablations confirm the bottleneck is frozen embedding quality, not model capacity or loss design.

3. Potential Risks and Challenges

Encoder quality dominates HCCS gain. Switching from CodeBERT to UniXcoder yields a ~ 5 -point MRR improvement, whereas architecture changes within the same encoder add < 1 MRR point. Regularization and loss ablations confirm that the frozen embeddings, not model capacity, are the performance bottleneck.

Retrieval and generation quality are not yet linked. Our evaluation measures whether the gold snippet is retrieved at rank k but does not verify that better retrieval leads to fewer hallucinations in generated code. This connection must be validated end-to-end.

Router and EFL remain unvalidated. The Type-Specific Context Router and Execution Feedback Loop are implemented but unbenchmarked. Their marginal contribution over HCCS-alone on downstream generation quality is unknown.

Scope limited to one benchmark and language. All experiments use RepoBench v1.1 Python `cross_file_first`. Generalization to other languages, RepoBench splits, or other benchmarks [11] is untested.

4. Risk Mitigation Plan

End-to-end generation evaluation. Our primary remaining deliverable is running the full HaluGuard pipeline—HCCS retrieval followed by DeepSeek-Coder-1.3B [10] generation—on the held-out test set and reporting Exact Match, Edit Similarity, and CodeBLEU against no-context, BM25, cosine, and gold-context baselines.

Type Router integration and ablation. We will compare HCCS-alone against HCCS + Router on the generation test set, measuring per-type improvement across the four hallucination categories (resource, naming, mapping, logic). The rule-based Router is already implemented and requires only evaluation harness integration.

Execution Feedback Loop evaluation. We will measure EFL retries by counting hallucination-type errors before and after EFL on at least 200 held-out test examples. If EFL inference is too slow, we will stratify and evaluate on a 50-example subset per category.

Addressing the encoder bottleneck. Time permitting, we will experiment with lightweight adapter layers on top of frozen UniXcoder to improve context discrimination without full fine-tuning of the 125M-parameter backbone, drawing on execution-feedback training ideas [9].

5. Individual Contributions

Sparsh Gupta: Built the data pipeline and baseline functions (Random, Jaccard, Edit Similarity, Cosine). Motivated the RepoBench pivot. Trained HCCS variants.

Alan Liang: Researched alternative training objectives, loss schedules, and regularization. Ran ablation studies on generalization.

Katherine Hancock: Generated all CodeBERT and UniXcoder embeddings. Trained HCCS variants. Wrote the initial draft of this report.

Daniel Ho: Explored MLP scorer improvements: alternative features, score distribution logging, warmup/scheduler tuning. Researched BigCode for pass@ k .

Freddie Liang: Implemented four scorer architectures across both encoders (8 experiments). Ran regularization (5 configs) and R@1-focused loss ablations (4 configs). Built the easy/hard evaluation harness. Demonstrated DualEncoder + listwise training fixes the hard@1 regression.

References

- [1] Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level code auto-completion systems. In *ICLR*, 2024.
- [2] Yuchen Tian et al. CodeHalu: Investigating code hallucinations in LLMs via execution-based verification. In *AAAI*, 2025.
- [3] Ziyao Zhang et al. LLM hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *arXiv:2409.20550*, 2025.

- [4] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of EMNLP*, pp. 1536–1547, 2020.
- [5] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. UniXcoder: Unified cross-modal pre-training for code representation. In *ACL*, pp. 7212–7225, 2022.
- [6] Wei Cheng et al. DRaCo: Dataflow-guided retrieval augmentation for repository-level code completion. In *ACL*, 2024.
- [7] Wei Liu et al. GraphCoder: Enhancing repository-level code completion via code context graph-based retrieval. *arXiv preprint*, 2024.
- [8] Huy N. Phan et al. RepoHyper: Search-expand-refine on semantic graphs for repository-level code completion. *arXiv:2403.06095*, 2024.
- [9] Jonas Gehring et al. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. In *ICML*, 2025.
- [10] Daya Guo, Qihao Zhu, Dejian Yang, et al. DeepSeek-Coder: When the large language model meets programming—the rise of code intelligence. *arXiv:2401.14196*, 2024.
- [11] Yangruibo Ding et al. CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion. *arXiv:2310.11248*, 2023.